⏱️ CATEGORY 9: Scheduling & Automation — Complete Deep Dive

9.1 `cron` & `crontab` — Syntax, Fields, Common Pitfalls ⚠️

🔷 What cron is in simple terms

`cron` is the classic Unix job scheduler — a daemon that wakes up every minute, checks its tables of scheduled jobs, and runs any that are due. It has been around since 1975. Despite being decades old, it runs the majority of scheduled tasks on Linux servers today.

🔷 How cron works internally

```
crond daemon starts at boot
          │
          ▼
Every minute: wakes up, reads all crontab files
          │
          ▼
For each entry: does current time match all 5 fields?
          │
     YES  │
          ▼
fork() → setuid(user) → exec(shell -c "command")
          │
          ▼
stdout/stderr → mailed to user (if MAILTO set and mail configured)
          │
          ▼
Go back to sleep until next minute

Crontab file locations:
  /var/spool/cron/crontabs/<username>   ← per-user crontabs (via crontab -
e)
  /etc/crontab                          ← system crontab (has extra user
field)
  /etc/cron.d/*                         ← drop-in crontab files (like
/etc/crontab format)
  /etc/cron.hourly/*                    ← scripts run hourly by run-parts
  /etc/cron.daily/*                     ← scripts run daily
  /etc/cron.weekly/*                    ← scripts run weekly
  /etc/cron.monthly/*                   ← scripts run monthly
```

🔷 Crontab syntax — The five time fields

```
┌──────────── minute        (0–59)
│ ┌──────────── hour          (0–23)
│ │ ┌──────────── day of month  (1–31)
│ │ │ ┌──────────── month         (1–12 or jan,feb,...dec)
│ │ │ │ ┌──────────── day of week   (0–7, where 0 and 7 = Sunday, or
sun,mon,...sat)
│ │ │ │ │
* * * * *  command to run

Special characters:
  *   = any value (every minute / every hour / etc.)
  ,   = list of values    → 1,15,30   (at minute 1, 15, and 30)
  -   = range of values   → 1-5       (minutes 1 through 5)
  /   = step values       → */15      (every 15 units)
  @   = special shorthand (see below)
```

🔷 Time field examples

```
# ── Every minute
* * * * *  /path/to/script.sh

# ── Every 5 minutes
*/5 * * * *  /path/to/script.sh

# ── Every 15 minutes
*/15 * * * *  /path/to/script.sh

# ── Every hour at minute 0
0 * * * *  /path/to/script.sh

# ── Every day at 2:30 AM
30 2 * * *  /path/to/script.sh

# ── Every day at midnight
0 0 * * *  /path/to/script.sh
# Equivalent: @midnight  /path/to/script.sh

# ── Every weekday (Mon-Fri) at 9 AM
0 9 * * 1-5  /path/to/script.sh

# ── Every Monday at 6 AM
0 6 * * 1  /path/to/script.sh
# Equivalent: 0 6 * * mon  /path/to/script.sh

# ── First day of every month at 1 AM
0 1 1 * *  /path/to/script.sh
```

```
# ── Every 6 hours
0 */6 * * *  /path/to/script.sh

# ── At 8 AM and 5 PM on weekdays
0 8,17 * * 1-5  /path/to/script.sh

# ── Every 10 minutes during business hours (9-17) on weekdays
*/10 9-17 * * 1-5  /path/to/script.sh

# ── Twice a year: Jan 1 and Jul 1 at midnight
0 0 1 1,7 *  /path/to/script.sh

# ── Special @ shortcuts
@reboot    /path/to/script.sh   # Run once at startup
@yearly    /path/to/script.sh   # = 0 0 1 1 *   (Jan 1 midnight)
@annually  /path/to/script.sh   # Same as @yearly
@monthly   /path/to/script.sh   # = 0 0 1 * *   (1st of month)
@weekly    /path/to/script.sh   # = 0 0 * * 0   (Sunday midnight)
@daily     /path/to/script.sh   # = 0 0 * * *   (midnight)
@midnight  /path/to/script.sh   # Same as @daily
@hourly    /path/to/script.sh   # = 0 * * * *   (top of every hour)

# ── Day of week: the OR trap ──────────────────────────────────────
# If BOTH day-of-month AND day-of-week are set (not *), cron runs when
EITHER matches

0 0 1 * 5  command
# Runs: on the 1st of every month  OR  every Friday
# NOT: "on Fridays that fall on the 1st of the month"!

# Workaround for "first Friday of month":
0 0 * * 5  [ "$(date +\%d)" -le 7 ] && /path/to/script.sh
```

🔷 `crontab` command — managing crontabs

```
# ── Per-user crontab management ───────────────────────────────────
crontab -e          # Edit your own crontab (validates before saving)
crontab -l          # List your crontab
crontab -r          # Remove your crontab (NO CONFIRMATION — dangerous!)

# Root: edit another user's crontab
sudo crontab -u alice -e
sudo crontab -u alice -l

# ── /etc/crontab — system-wide crontab ────────────────────────────
cat /etc/crontab
# SHELL=/bin/sh
# PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
#
# m  h  dom  mon  dow  user   command
```

```
# 17 *    *    *    *   root   cd / && run-parts --report /etc/cron.hourly
# 25 6    *    *    *   root   test -x /usr/sbin/anacron || run-parts --
report /etc/cron.daily
#
# Note: /etc/crontab has an EXTRA "user" field before the command
# User crontabs (crontab -e) do NOT have this field

# ── /etc/cron.d/ — drop-in files (same format as /etc/crontab) ──────
cat /etc/cron.d/myapp
# SHELL=/bin/bash
# PATH=/usr/local/bin:/usr/bin:/bin
# MAILTO=ops@example.com
#
# */5 * * * *  myapp  /usr/local/bin/health-check >> /var/log/health.log
2>&1

# ── /etc/cron.daily/ etc. — script drop-ins ──────────────────────────
# Place an executable script here — no crontab syntax needed
cat /etc/cron.daily/backup-db
#!/bin/bash
set -euo pipefail
pg_dump mydb > /backup/db_$(date +%Y%m%d).sql
find /backup/ -name "db_*.sql" -mtime +7 -delete

chmod +x /etc/cron.daily/backup-db
# IMPORTANT: Must not have a dot in filename — run-parts SKIPS files with
dots!

# Preview what run-parts would execute
run-parts --test /etc/cron.daily
```

🔷 Environment variables in cron

```
# Cron's minimal default environment (not your shell!):
# HOME=/root (or user's home)
# SHELL=/bin/sh            ← /bin/sh, NOT bash!
# PATH=/usr/bin:/bin       ← very restricted!
# MAILTO=username          ← where to send output

# ── Fix: set environment at top of crontab ──────────────────────────
crontab -e
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
MAILTO=ops@example.com
HOME=/home/alice

0 2 * * *  /usr/local/bin/backup.sh

# ── Fix: always use absolute paths ──────────────────────────────────
# BAD:
```

```
0 2 * * *  backup.sh            # Might not be in cron's PATH!
# GOOD:
0 2 * * *  /usr/local/bin/backup.sh

# ── MAILTO — controlling output ──────────────────────────────────
# By default, cron mails stdout+stderr to user
# If mail not configured: output is SILENTLY LOST

MAILTO=""                       # Suppress all mail
0 2 * * *  /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
# Always redirect explicitly when MAILTO is empty

# ── Locking: prevent overlapping runs ────────────────────────────
# Problem: if job takes 7 min and runs every 5 min → pile-up
*/5 * * * *  flock -n /tmp/myjob.lock /usr/local/bin/myjob.sh
# flock -n = non-blocking: skip run if lock already held
```

🔷 Testing cron jobs

```
# Test with cron's actual environment
env -i HOME=/root LOGNAME=root USER=root \
    SHELL=/bin/sh PATH=/usr/bin:/bin \
    /bin/sh -c '/usr/local/bin/myjob.sh'

# Watch cron logs live
journalctl -u cron -f
# Or:
tail -f /var/log/syslog | grep CRON
# Mar 10 02:30:01 server CRON[12345]: (alice) CMD
(/usr/local/bin/backup.sh)
# Mar 10 02:30:01 server CRON[12345]: (CRON) info (No MTA installed,
discarding output)
#                                                  ↑ Use log redirect,
not MAILTO!

# Access control
cat /etc/cron.allow   # If exists: ONLY listed users can use cron
cat /etc/cron.deny    # Listed users cannot use cron
```

🔷 Short crisp interview answer

> "Cron reads five time fields (minute, hour, day-of-month, month, day-of-week) plus a command. The biggest production pitfalls: cron runs with a minimal environment — PATH is `/usr/bin:/bin`, shell is `/bin/sh`, and `~/.bashrc` is never sourced — so I always use absolute paths and set `SHELL=/bin/bash` and `PATH` at the top of the crontab. I always redirect output with `>> /var/log/job.log 2>&1` because unhandled output silently disappears if `MAILTO` isn't

> configured. For jobs that must not overlap, `flock -n /tmp/job.lock command` skips the run if the previous one is still going."

⚠ Gotchas

```
# GOTCHA 1: PATH is minimal — commands not found
0 2 * * *  node /app/script.js   # Fails! node in /usr/local/bin, not in
cron's PATH
# Fix: set PATH at top of crontab or use /usr/local/bin/node

# GOTCHA 2: Shell is /bin/sh — bash syntax breaks
0 2 * * *  [[ -f /tmp/file ]] && process.sh  # [[ is bash-only, fails in
sh!
# Fix: SHELL=/bin/bash at top of crontab

# GOTCHA 3: % means newline in crontab
0 2 * * *  date "+%Y-%m-%d"   # BREAKS — % is newline to cron!
# Fix: escape with \%
0 2 * * *  date "+\%Y-\%m-\%d"
# Or: put command in a script file (scripts don't need escaping)

# GOTCHA 4: Day-of-month AND day-of-week is OR not AND
0 0 1 * 5  cmd   # Runs on 1st of month OR every Friday — not first Friday
only!

# GOTCHA 5: No MTA = output silently lost — ALWAYS redirect to log

# GOTCHA 6: Scripts in cron.daily with dot extension are SKIPPED
/etc/cron.daily/backup.sh   # SKIPPED by run-parts!
/etc/cron.daily/backup      # Runs correctly

# GOTCHA 7: @reboot fires before network is up
@reboot  /usr/local/bin/check-endpoint.sh   # May fail — network not ready
# Fix: add sleep or use systemd service with After=network.target

# GOTCHA 8: Overlapping jobs pile up
*/5 * * * *  /slow/job.sh   # Job takes 7 min → two copies running!
# Fix: flock -n /tmp/job.lock /slow/job.sh

# GOTCHA 9: crontab -r has no confirmation
crontab -r   # Instantly deletes entire crontab — no undo!
# Safe habit: crontab -l > crontab.backup before any changes
```

9.2 `at` & `batch` — One-Off Job Scheduling

🔷 What `at` and `batch` are

While cron schedules **recurring** jobs, `at` schedules **one-time** jobs at a specific future time. `batch` runs jobs when **system load drops** below a threshold — useful for resource-intensive tasks that should run opportunistically without degrading the live system.

🔷 How `at` works internally

```
User submits: at 3pm tomorrow
                |
                ▼
        atd daemon receives job
                |
        Job stored in: /var/spool/at/
        (a shell script with captured current environment)
                |
        Every minute: atd checks for due jobs
                |
        Is any job's run time <= now?
           YES  |
                ▼
        fork() -> setuid(user) -> exec(job script)
                |
        stdout/stderr -> mailed to user (like cron)

Key difference from cron:
  - Job runs ONCE, then deleted from /var/spool/at/
  - Captures YOUR CURRENT environment at submission time
    (unlike cron which uses a minimal environment)
```

🔷 `at` — schedule a one-time job

```bash
# Install and enable
sudo apt install at
sudo systemctl enable --now atd

# ── Interactive: type commands, then Ctrl+D ──────────────────────
at 3pm
# at> /usr/local/bin/backup.sh
# at> <Ctrl+D>
# job 1 at Mon Mar 10 15:00:00 2026

# ── Pipe command to at ────────────────────────────────────────────
echo "/usr/local/bin/backup.sh" | at 3pm

# ── Heredoc (multiple commands) ───────────────────────────────────
at 2:30am tomorrow << 'EOF'
#!/bin/bash
cd /app
./generate-report.sh > /tmp/report.html
```

```
mail -s "Daily Report" ops@example.com < /tmp/report.html
EOF

# ── Time specification — very flexible ──────────────────────────────
at 15:00                  # 3:00 PM today
at 3pm                    # Same
at 3:30pm                 # 3:30 PM
at now + 5 minutes        # 5 minutes from now
at now + 2 hours          # 2 hours from now
at now + 3 days           # 3 days from now
at 3pm tomorrow           # Tomorrow at 3 PM
at 3pm next monday        # Next Monday at 3 PM
at 3pm friday             # This Friday
at 3pm March 15           # March 15 at 3 PM
at noon                   # 12:00 PM today
at midnight               # 12:00 AM tonight
at teatime                # 4:00 PM (British tradition!)
at now                    # Immediately (good for testing)

# ── Managing at jobs ────────────────────────────────────────────────
atq                       # List pending jobs
# 3  Mon Mar 11 03:00:00 2026 a alice
# 4  Mon Mar 11 15:30:00 2026 a alice
# 5  Mon Mar 11 23:59:00 2026 b root   ← 'b' = batch queue

at -c 3                   # View job #3's script (debugging)
atrm 3                    # Cancel job #3
atrm 3 4 5                # Cancel multiple
at -d 3                   # Same as atrm

# ── Key advantage: at captures current environment ──────────────────
echo $JAVA_HOME           # /usr/lib/jvm/java-11
echo $DATABASE_URL        # postgres://localhost/mydb
# These are SAVED in the at job script automatically
# Test: at +5 minutes <<< "printenv > /tmp/at_env.txt"

# ── Practical examples ──────────────────────────────────────────────

# Schedule a maintenance restart
at 2am Sunday <<< "sudo systemctl restart myapp"

# Delayed deployment with cancellation window
at now + 10 minutes <<< "systemctl start new-service"
# If something wrong in 10 min: atrm <jobid> to cancel!

# Redirect output (no MAILTO by default)
at 3pm <<< "/usr/local/bin/job.sh >> /var/log/job.log 2>&1"

# ── Access control ──────────────────────────────────────────────────
# /etc/at.allow — only listed users can use at (deny-by-default)
# /etc/at.deny  — listed users cannot use at
# Same logic as cron.allow / cron.deny
```

🔷 `batch` — Run when system load drops

```
# batch = at but waits for system load < 1.5 (default threshold)
# Same syntax as at, but deferred until system is quiet

batch << 'EOF'
/usr/local/bin/generate-thumbnails.sh /data/images/
EOF
# job 6 at Mon Mar 10 15:00:00 2026  (runs when load drops, not at 15:00!)

# batch is equivalent to:
at -q b -f myscript.sh now    # Queue 'b' = batch queue in atd

# View batch jobs (show in atq with 'b' in queue column)
atq
# 6  Mon Mar 10 15:00:00 2026 b alice  ← batch job, waiting for low load

# Good use cases for batch:
batch <<< "ffmpeg -i /data/input.mp4 -codec:a libmp3lame /data/output.mp3"
batch <<< "mysql -e 'ANALYZE TABLE large_table;' mydb"
batch <<< "tar -czf /backup/archive.tar.gz /data/large-directory/"
```

🔷 Short crisp interview answer

> "`at` schedules a command to run once at a specific future time — unlike cron which is for recurring jobs. `echo 'command' | at 3pm tomorrow` schedules it; `atq` lists pending jobs; `atrm <id>` cancels one. A key advantage over cron: `at` captures your current environment at submission time, so environment variables, PATH, and shell settings are all preserved. `batch` is the same as `at` but only runs when system load drops below 1.5 — useful for scheduling heavy computation like image processing or database maintenance to happen opportunistically when the system is quiet."

⚠ Gotchas

```
# GOTCHA 1: atd must be running
systemctl status atd   # If not running, at commands appear to work but
jobs never execute

# GOTCHA 2: Output silently lost without redirect
at 3pm <<< "/job.sh"             # stdout/stderr mailed if MAILTO works
at 3pm <<< "/job.sh > /var/log/job.log 2>&1"  # Safer — explicit redirect

# GOTCHA 3: atq shows submission time for batch, not run time
# batch jobs show when they were submitted, not when they'll actually run

# GOTCHA 4: at time parsing can be ambiguous
at 3           # Potentially ambiguous — use 3am, 3pm, or 03:00
```

```
at 15:00       # Unambiguous

# GOTCHA 5: Timezone awareness
# at uses local system timezone
TZ=UTC at 02:00 <<< "/usr/local/bin/job.sh"  # Force UTC if needed
```

9.3 `systemd` Timers — Modern Replacement for Cron

🔷 What systemd timers are

Systemd timers are the **modern recommended way** to schedule tasks on systemd-based Linux. Unlike cron, timers are full systemd units with dependency management, resource limits, journald logging, and structured status reporting.

🔷 How systemd timers work

```
Timer unit (.timer) ——— "Activates" (starts) ———► Service unit
(.service)
    │                                                    │
    │  when to run                                  what to run
    │                                                    │
    ▼                                                    ▼
Timer resets, waits                              journald logs everything
for next trigger                                 (structured, queryable)

Key architecture: ALWAYS a pair of files:
  mybackup.timer    ← when to run
  mybackup.service  ← what to run

The timer activates the service.
The service does the actual work.
```

🔷 Timer types

```
# Realtime (calendar) timers — like cron, fire at specific times
# Uses: OnCalendar=

# Monotonic timers — fire relative to events
# Uses:
#   OnBootSec=         → N seconds after system boot
#   OnStartupSec=      → N seconds after systemd started
#   OnUnitActiveSec=   → N seconds after this timer last activated
#   OnUnitInactiveSec= → N seconds after the service last deactivated
```

🔷 Creating a systemd timer — step by step

```
# Step 1: Create the service unit (/etc/systemd/system/db-backup.service)
[Unit]
Description=Daily PostgreSQL backup
After=postgresql.service
Wants=postgresql.service

[Service]
Type=oneshot
User=postgres
Group=postgres
ExecStart=/usr/local/bin/backup-postgres.sh

# Resource limits — impossible with cron!
MemoryLimit=512M
CPUQuota=50%

# Logging goes to journald automatically
StandardOutput=journal
StandardError=journal

# Kill if takes too long
TimeoutStartSec=3600

# Environment
EnvironmentFile=/etc/db-backup.env

# Step 2: Create the timer unit (/etc/systemd/system/db-backup.timer)
[Unit]
Description=Daily database backup timer

[Timer]
# Run daily at 2:30 AM
OnCalendar=*-*-* 02:30:00

# If system was off at 2:30 AM, run when it comes back up
Persistent=true

# Randomize start within 15 min (prevents thundering herd on fleet)
RandomizedDelaySec=15min

# Which service to activate (default: same name with .service)
Unit=db-backup.service

[Install]
WantedBy=timers.target

# Step 3: Enable and start
sudo systemctl daemon-reload          # Always required after
```

```
creating/editing unit files!
sudo systemctl enable --now db-backup.timer

# Step 4: Verify
systemctl list-timers --all           # Shows NEXT and LAST trigger times
systemctl status db-backup.timer
journalctl -u db-backup.service       # View logs from all past runs
```

🔷 OnCalendar syntax — the cron equivalent

```
# Format: DayOfWeek Year-Month-Day Hour:Minute:Second
# Wildcards:  * = any,  , = list,  .. = range,  / = step

# ── Common OnCalendar patterns ──────────────────────────────────────
OnCalendar=*-*-* *:*:00          # Every minute
OnCalendar=*-*-* *:0/5:00        # Every 5 minutes
OnCalendar=hourly                 # Every hour at :00
OnCalendar=daily                  # Every day at 00:00:00
OnCalendar=*-*-* 02:30:00        # Every day at 2:30 AM
OnCalendar=Mon..Fri *-*-* 09:00:00  # Every weekday at 9 AM
OnCalendar=Mon *-*-* 08:00:00    # Every Monday at 8 AM
OnCalendar=weekly                 # Sundays at midnight
OnCalendar=monthly                # 1st of month at midnight
OnCalendar=*-01,04,07,10-01 00:00:00  # Quarterly
OnCalendar=*-*-* 00/6:00:00      # Every 6 hours
# Multiple lines are allowed — fires on any match:
OnCalendar=*-*-* 08:00:00
OnCalendar=*-*-* 20:00:00        # Twice a day: 8 AM and 8 PM
OnCalendar=Mon..Fri *-*-* 09..17:0/15:00  # Every 15 min, business hours,
weekdays
OnCalendar=2026-12-31 23:59:00   # One-off specific date

# Special keywords
OnCalendar=hourly
OnCalendar=daily
OnCalendar=weekly
OnCalendar=monthly
OnCalendar=annually

# ── Test your expression BEFORE deploying ──────────────────────────
systemd-analyze calendar "*-*-* 02:30:00"
# Original form:   *-*-* 02:30:00
# Normalized form: *-*-* 02:30:00
# Next elapse:     Mon 2026-03-11 02:30:00 IST
#                  Tue 2026-03-12 02:30:00 IST
#                  Wed 2026-03-13 02:30:00 IST  ...

systemd-analyze calendar "Mon..Fri *-*-* 09:00:00"
# Shows next 5 weekday mornings at 9 AM — validates before you deploy
```

```
# ── Monotonic examples ──────────────────────────────────────────
[Timer]
OnBootSec=5min                  # Run 5 minutes after every boot
OnUnitActiveSec=30min           # Then every 30 minutes after that

[Timer]
OnBootSec=10min                 # Startup health check
OnUnitActiveSec=5min            # Then every 5 minutes
```

🔷 Managing and monitoring timers

```
# ── List timers ─────────────────────────────────────────────────
systemctl list-timers
# NEXT                        LEFT     LAST                        PASSED  UNIT
# Mon 2026-03-11 02:30:00  11h left Sun 2026-03-10 02:30:01  12h ago db-
backup.timer
# Mon 2026-03-11 00:00:00  8h left  Sun 2026-03-10 00:00:02  15h ago
logrotate.timer

systemctl list-timers --all    # Include inactive timers

# ── Status and logs ─────────────────────────────────────────────
systemctl status db-backup.timer    # When will it next fire?
systemctl status db-backup.service  # How did last run go?

journalctl -u db-backup.service           # Full log history
journalctl -u db-backup.service --since today
journalctl -u db-backup.service -n 50     # Last 50 lines
journalctl -u db-backup.service -f        # Follow live

# ── Lifecycle management ────────────────────────────────────────
sudo systemctl enable db-backup.timer     # Enable at boot
sudo systemctl disable db-backup.timer    # Disable
sudo systemctl start db-backup.timer      # Start timer now
sudo systemctl stop db-backup.timer       # Stop timer

# After editing unit files:
sudo systemctl daemon-reload
sudo systemctl restart db-backup.timer

# ── Trigger service manually for testing ────────────────────────
sudo systemctl start db-backup.service    # Bypass timer, run service now
journalctl -u db-backup.service -f        # Watch it run

# ── Transient timers with systemd-run (like 'at' with systemd) ──────
sudo systemd-run --on-active=5min /usr/local/bin/maintenance.sh
sudo systemd-run --on-calendar="*-*-* 03:00:00" /usr/local/bin/cleanup.sh
sudo systemd-run \
    --unit=my-heavy-job \
    --uid=alice \
```

```
    --property=MemoryLimit=1G \
    --property=CPUQuota=25% \
    /usr/local/bin/heavy-job.sh
# NOTE: transient units are lost on reboot — write unit files for
persistence
```

🔷 Advanced timer features

```
# ── Persistent — catch missed runs ──────────────────────────────
[Timer]
OnCalendar=daily
Persistent=true
# If system was off at trigger time, run on next boot
# Stores last-run time in /var/lib/systemd/timers/
# Like anacron, but built into systemd

# ── RandomizedDelaySec — prevent thundering herd ────────────────
[Timer]
OnCalendar=daily
RandomizedDelaySec=30min
# 500 servers all set to run at 2 AM:
# Without: all hit database at exactly 2:00:00
# With: spread randomly across 2:00–2:30

# ── AccuracySec — power efficiency on laptops ───────────────────
[Timer]
OnCalendar=hourly
AccuracySec=10min     # Can fire up to 10 min late to batch with other
wakeups
# Default: AccuracySec=1min

# ── WakeSystem — wake from suspend ──────────────────────────────
[Timer]
OnCalendar=*-*-* 02:00:00
WakeSystem=true        # Wake laptop from sleep to run this timer

# ── Multiple triggers in one timer ──────────────────────────────
[Timer]
OnCalendar=Mon *-*-* 09:00:00
OnCalendar=Wed *-*-* 09:00:00
OnCalendar=Fri *-*-* 09:00:00    # Fires Mon, Wed, Fri at 9 AM
```

🔷 Real-world examples

```
# ── Example: certificate renewal (certbot) ──────────────────────

# /etc/systemd/system/certbot-renewal.service
```

```
[Unit]
Description=Certbot Renewal
After=network.target

[Service]
Type=oneshot
User=root
ExecStart=/usr/bin/certbot renew --quiet --agree-tos
ExecStartPost=/bin/systemctl reload nginx
StandardOutput=journal
StandardError=journal

# /etc/systemd/system/certbot-renewal.timer
[Unit]
Description=Run certbot twice daily

[Timer]
OnCalendar=*-*-* 00,12:00:00
RandomizedDelaySec=1h
Persistent=true

[Install]
WantedBy=timers.target

# ── Example: cleanup job with resource limits ──────────────────────────

# /etc/systemd/system/temp-cleanup.service
[Unit]
Description=Clean temporary files

[Service]
Type=oneshot
ExecStart=/usr/bin/find /tmp -type f -atime +7 -delete
ExecStart=/usr/bin/find /var/tmp -type f -atime +30 -delete
CPUQuota=10%
IOSchedulingClass=idle
MemoryLimit=256M
Nice=19

# /etc/systemd/system/temp-cleanup.timer
[Timer]
OnCalendar=*-*-* 03:00:00
Persistent=true
RandomizedDelaySec=30min

[Install]
WantedBy=timers.target

sudo systemctl daemon-reload
sudo systemctl enable --now temp-cleanup.timer

# ── Example: health check every 5 minutes ──────────────────────────────

# /etc/systemd/system/health-check.timer
```

```
[Timer]
OnBootSec=2min
OnUnitActiveSec=5min

[Install]
WantedBy=timers.target
```

🔷 systemd timers vs cron

```
Feature                 | cron                          | systemd timer
————————————————————————|———————————————————————————————|————————————————————————
————————
Logging                 | syslog or email (lossy)       | journald (structured,
always)
Dependencies            | None (always runs)            | After=, Requires=,
Wants=
Resource limits         | None                          | CPUQuota=,
MemoryLimit=, etc.
Missed jobs             | Lost (unless anacron)         | Persistent=true
Environment             | Minimal (must configure)      | Defined in unit file
Output handling         | Email (often not set up)      | journald (always
works)
Manual trigger          | Run script directly           | systemctl start
svc.service
Thundering herd         | All fire at same second       | RandomizedDelaySec=
Boot-relative timing    | @reboot (once only)           | OnBootSec=
(repeating)
Complexity              | Simple, one line              | Two files, more setup
Portability             | Any Unix system               | systemd systems only

Use cron for:          Simple tasks, portability, user-level scheduling
Use systemd for:       Production services, resource limits, dependencies,
logging
```

🔷 Short crisp interview answer

> "Systemd timers are pairs of units: a `.timer` that says *when* and a `.service` that says *what*. The key advantages over cron: logs go to journald — structured, queryable, never lost — and you can set `MemoryLimit=` and `CPUQuota=` on scheduled jobs. `Persistent=true` catches missed jobs like anacron, and `RandomizedDelaySec=` spreads thundering-herd problems across a fleet. `OnCalendar=*-*-* 02:30:00` equals `30 2 * * *` in cron — test with `systemd-analyze calendar`. I use `systemctl list-timers --all` to see next run times, and `journalctl -u myservice.service` to see logs from every past run."

⚠ Gotchas

```
# GOTCHA 1: Must daemon-reload after creating/editing unit files
sudo nano /etc/systemd/system/myapp.timer
sudo systemctl daemon-reload    # REQUIRED — without this, changes are
invisible!

# GOTCHA 2: Timer and service must have matching names (or use Unit=)
# myapp.timer automatically activates myapp.service
# Different name? Add:  Unit=different-service.service  in [Timer] section

# GOTCHA 3: Enable the timer, NOT the service
sudo systemctl enable myapp.timer    # Correct — timer fires on schedule
sudo systemctl enable myapp.service  # Wrong — runs at boot every boot

# GOTCHA 4: Use Type=oneshot for scheduled tasks
[Service]
Type=simple    # Wrong — systemd thinks it's a long-running daemon
Type=oneshot   # Correct — systemd waits for ExecStart to exit

# GOTCHA 5: OnUnitActiveSec vs OnCalendar
# OnCalendar: fires at ABSOLUTE times (2 AM regardless of last run)
# OnUnitActiveSec: N seconds AFTER LAST ACTIVATION (sliding window)
# 30-second job + OnUnitActiveSec=1min = effectively runs every 1.5 min
total

# GOTCHA 6: Transient systemd-run units are lost on reboot
sudo systemd-run --on-calendar="daily" /usr/local/bin/myjob.sh
# Disappears after reboot — write proper unit files for persistence

# GOTCHA 7: OnCalendar day-of-week capitalization
OnCalendar=mon *-*-* 09:00:00    # May not work
OnCalendar=Mon *-*-* 09:00:00    # Correct — capitalize
OnCalendar=Monday *-*-* 09:00:00 # Full name also works

# GOTCHA 8: Persistent=true requires writable /var/lib/systemd/timers/
# On read-only root filesystem, Persistent=true fails silently
```

🏆 Category 9 — Complete Mental Model

```
SCHEDULING TOOL DECISION TREE
═══════════════════════════════

Need to schedule a task?
          │
          ├─ One-time, future time? ──────────────► at
          │   └─ Run when system is quiet/idle?        └─► batch
          │
          ├─ Recurring, simple, portable? ────────► cron
          │   (or system without systemd)
```

```
            │
            └─ Recurring, on modern systemd system? ──► systemd timer
(preferred)
               Need any of:
                 resource limits?    ──────────────────► systemd timer
                 service dependency? ──────────────────► systemd timer
                 catch missed runs?  ──────────────────► systemd timer
(Persistent=true)
                 spread across fleet?──────────────────► systemd timer
(RandomizedDelaySec)
                 structured logging? ──────────────────► systemd timer
(journald)

QUICK REFERENCE:
═══════════════

cron:
  crontab -e                      # Edit user crontab
  crontab -l                      # List crontab
  /etc/cron.d/                    # System drop-ins (with user field)
  /etc/cron.daily/                # Scripts — no cron syntax, no
extensions!
  grep CRON /var/log/syslog       # View cron activity

at / batch:
  echo "cmd" | at 3pm             # Schedule one-off
  at -f script.sh 3pm             # From file
  atq                             # List pending jobs
  atrm 3                          # Cancel job #3
  at -c 3                         # Show job #3 script
  echo "cmd" | batch              # Run when load drops

systemd timer:
  systemd-analyze calendar "expr" # Test OnCalendar expression
  systemctl list-timers --all     # All timers + next/last run times
  systemctl status mytimer.timer  # Timer status
  systemctl start myservice.service # Manual trigger (bypasses timer)
  journalctl -u myservice.service  # Full log history
  systemd-run --on-active=5min cmd # Transient one-off
  systemctl daemon-reload          # Always after editing unit files!
```

⚠ Master Gotcha List

#	Gotcha	Fix
1	Cron PATH = `/usr/bin:/bin` only	Set `PATH=...` at top of crontab or use absolute paths
2	Cron shell = `/bin/sh` not bash	Set `SHELL=/bin/bash` at top of crontab
3	`%` in crontab = newline	Escape as `\%` or put command in a script file

#	Gotcha	Fix
4	DOM + DOW = OR not AND	Use script-side date check for AND logic
5	No MTA = output silently lost	`MAILTO=""` + redirect to log file
6	`cron.daily` scripts with dots skipped	Remove extension from scripts in cron directories
7	`@reboot` fires before network	Add `sleep 30` or use systemd service `After=network.target`
8	Overlapping cron jobs pile up	`flock -n /tmp/lock.file command`
9	`crontab -r` — no confirmation	`crontab -l > backup` before changes
10	`atd` not running = silent failure	`systemctl status atd` before using `at`
11	No `daemon-reload` after editing timer	Always `systemctl daemon-reload`
12	`Type=simple` on scheduled tasks	Use `Type=oneshot` for tasks that run and exit
13	`OnUnitActiveSec` vs `OnCalendar`	`OnCalendar` for fixed times, `OnUnitActiveSec` for intervals
14	`systemd-run` timers lost on reboot	Write proper `.timer` + `.service` unit files

🔥 Top Interview Questions

Q1: A cron job works manually but fails when run by cron. Why?

> Almost always an environment issue. Cron runs with a minimal environment: `PATH=/usr/bin:/bin`, shell is `/bin/sh` (not bash), and `~/.bashrc` is never sourced. Fix: set `SHELL=/bin/bash` and full `PATH` at the top of the crontab, use absolute paths for all commands, and redirect output with `>> /var/log/job.log 2>&1` to see error messages — otherwise they disappear if `MAILTO` isn't configured.

Q2: How do you prevent a cron job from running multiple overlapping instances?

> Wrap with `flock`: `*/5 * * * * flock -n /tmp/myjob.lock /usr/local/bin/myjob.sh`. The `-n` flag is non-blocking — if the lock is held (previous run still going), the new invocation immediately exits instead of waiting. The lockfile is cleaned up automatically when the process exits.

Q3: What's the difference between `at` and `batch`?

> Both schedule one-time jobs. `at` runs at a specific absolute time you provide. `batch` runs when system load average drops below 1.5 — it queues immediately but atd only dispatches it when the system is quiet. Use `batch` for heavy CPU or I/O tasks (image processing, large file compression, database maintenance) that should happen opportunistically without impacting live traffic.

Q4: What are the key advantages of systemd timers over cron?

> Four main advantages: (1) **Logging** — output goes to journald, queryable with `journalctl -u service.service`, never lost to missing MAILTO. (2) **Resource limits** — `CPUQuota=25%` and `MemoryLimit=512M` in the service unit prevent scheduled tasks from overwhelming the system. (3) **Dependencies** — `After=postgresql.service` means the backup waits for the database to be running. (4) **Missed job recovery** — `Persistent=true` tracks last-run time and runs on next boot if the system was off at the scheduled time.

Q5: How do you test a systemd timer's `OnCalendar` expression?

> `systemd-analyze calendar "Mon..Fri *-*-* 09:00:00"` — it shows the normalized form and the next 5 trigger times. This validates syntax and shows exactly when it will fire before you deploy it. For the overall fleet view, `systemctl list-timers --all` shows LAST and NEXT trigger times for every timer on the system.

Q6: 500 servers all have a cron job at 2 AM. What's the problem and solution?

> Thundering herd — all 500 servers hit the same database or API at exactly 2:00:00 AM, creating a massive traffic spike. With cron, the workaround is `sleep $((RANDOM % 1800))` at the start of the script to randomize the start time within 30 minutes. With systemd timers, the built-in solution is `RandomizedDelaySec=30min` in the `[Timer]` section — systemd automatically and reproducibly spreads the timer across a 30-minute window without any script changes.

This document covers all 3 topics in Category 9: Scheduling & Automation — from cron fundamentals and pitfalls, through one-off scheduling with at/batch, to modern systemd timers with dependency management, resource controls, and structured logging.